

01 Agent定义、边界与适用场景

一、这一题在面试里到底考什么

二、什么是 Agent：从会回答到会完成任务

1. 围绕目标运行
2. 在执行过程中做决策
3. 借助外部能力推进任务
4. 持续运行直到终止条件满足

三、什么不算 Agent：边界比定义更重要

1. 普通 chatbot 不一定是 agent
2. 固定流程的 workflow 不一定是 agent
3. 只有 function calling 不足以自动成为 agent
4. 有记忆不等于 agent

四、为什么 Agent 会火：它到底解决了什么问题

场景一：任务路径事先不确定

场景二：任务需要多轮环境交互

场景三：任务规模超过单轮上下文

场景四：任务要求一定程度的自动化闭环

五、Agent 的核心构成：面试一定要会讲的最小框架

1. Goal / Task
2. Policy / Planner
3. Tools / Environment
4. State / Memory
5. Guardrails / Governance
6. Evaluator / Verifier

六、Agent 的自治程度：不要把全自动当成唯一答案

1. Assisted Agent
2. Semi-Autonomous Agent
3. Autonomous Agent
4. Open-Ended Agent

七、Agent 适合什么场景，不适合什么场景

适合的场景

1. 路径不固定、需要探索
2. 明显依赖中间观察结果
3. 任务可以被拆成阶段，但阶段之间仍需动态协调
4. 任务价值高于控制成本

不适合的场景

1. 路径清晰且稳定
2. 错误代价极高但又难以完全约束
3. 数据与接口不稳定、但系统又不能容错
4. 只是为了看起来高级

九、常见误区与纠偏

误区一：用了工具就是 agent

误区二：agent 一定比 workflow 高级

误区三：agent 就是把 prompt 写复杂一点

误区四：只要模型够强，就不需要系统设计

误区五：agent 的核心是思维链

十、Agent 的最小运行闭环

十一、高频面试题与参考回答

题 1：什么是 Agent？和普通 LLM 应用有什么不同？

题 2：带 function calling 的聊天机器人算不算 agent？

题 3：Workflow 和 Agent 的区别是什么？

题 4：Agent 一定需要工具吗？

题 5：什么时候你不会建议上 Agent？

题 6：为什么大家总说 agent 难做？

题 7：Agent 的核心组件有哪些？

题 8：Agent 和 RAG 的关系是什么？

题 9：Agent 和多 agent 是一回事吗？

题 10：怎样判断一个需求是否应该 agent 化？

十二、面试速记版

参考资料与延伸阅读

一、这一题在面试里到底考什么

什么是 Agent 几乎是所有 Agent 面试的起手题，但它真正考察的绝不是百科式定义，而是候选人有没有建立清晰的边界意识。面试官通常借这个问题判断三件事：第一，你是否真的理解 agent 和 chatbot、workflow、tool-calling app 的区别；第二，你是否知道 agent 什么时候有必要、什么时候没有必要；第三，你是否具备将看起来很智能的概念落到可上线、可治理、可控风险的工程能力。

因为近两年大家都在讲智能体，很多简历也会写做过 agent，但实际做的可能只是：给模型挂了两个工具，用 function calling 调一下搜索，然后按固定顺序执行。这样的系统不一定错，很多时候还更实用，但它未必就是一个典型意义上的 agent。面试官之所以要追问定义，不是为了抠字眼，而是为了确认：你有没有把自治多步决策基于观察更新后续动作面向目标而非单次回答这些关键特征理解到位。

从校招生视角看，最容易犯的错误有两个。**第一个错误是把任何用了大模型和工具的系统都叫 agent，导致边界完全失焦。第二个错误是把 agent 神化，认为 agent 比 workflow 更高级、更智能、一定更值得做。**实际上，很多官方实践都反复强调：当任务流程清晰、约束强、结果要求稳定时，workflow 往往更合适；agent 只有在路径不确定、需要动态规划或环境交互时，才真正发挥价值。这意味着，面试官心里理想的回答，不是 agent 是一种很强的 AI 系统，而是 agent 是在一定自治程度下围绕目标进行多步决策与动作执行的系统，但是否值得引入，要看任务的不确定性和工程约束。

如果面试官问什么是 Agent？，可以这样答：

Agent 本质上是一个围绕目标运行的系统，而不只是一次输入输出的模型调用。它通常会在多步执行中基于当前观察结果动态决定下一步动作，并通过工具或外部环境持续推进任务完成。和普通 chatbot 相比，Agent 更强调目标驱动、多步决策、环境交互和状态持续；和 workflow 相比，Agent 的关键差异在于运行时有更高的决策自由度，而不是所有步骤都预先写死。

在工程上，我会把 Agent 看成由目标、策略/规划、工具、状态/记忆、控制约束和校验机制组成。是否应该用 Agent，要看任务路径是否不确定、是否需要多轮环境交互，以及引入自治后能否接受成本、延迟和风险。

二、什么是 Agent：从会回答到会完成任务

先给一个足够工程化的定义：**Agent 是一种围绕任务目标运行、能够在执行过程中根据中间观察结果决定后续动作，并借助工具或外部环境持续推进任务完成的系统。**

这个定义里有四个关键词，面试时最好展开：

1. 围绕目标运行

普通聊天模型更像给定输入，生成输出；agent 更像给定目标，持续推进直到结束。

比如把这份周报润色一下更接近一次性生成任务；而帮我收集竞品价格、汇总对比并发到指定群里就更接近 agent 任务，因为它天然包含多个步骤和中间状态。

2. 在执行过程中做决策

Agent 不是只在开始时被动生成一段答案，而是在每一步都可能根据环境反馈改变路线。

例如它先搜索资料，再发现资料不够，就继续检索；发现一个 API 返回错误，就换另一种查询方式；在工具结果与预期不符时，可能回退、重试或重新规划。

3. 借助外部能力推进任务

这通常表现为工具调用，但本质是 agent 不只依赖参数里的知识，还会主动使用模型外的能力：检索、数据库、网页、代码解释器、文件系统、企业系统 API、浏览器、手机、桌面环境等。

这也是为什么 agent 与纯对话系统的工程重心完全不同：它不只是生成文字，而是要和现实世界产生交互。

4. 持续运行直到终止条件满足

Agent 往往不是一问一答结束，而是存在一个运行循环：思考、选择动作、执行、观察、更新状态、再决策，直到任务完成、失败、超时、预算耗尽或被人工中断。

所以一旦系统进入 agent 范式，状态管理、终止条件、审计与回放能力就变得非常关键。

三、什么不算 Agent：边界比定义更重要

很多候选人会在这一题上吃亏，不是因为不会定义，而是因为不会说什么不算。下面给出面试中最常见的边界问题。

1. 普通 chatbot 不一定是 agent

如果系统只是把用户问题喂给模型，然后直接返回一段答案，没有多步决策，没有工具使用，也没有目标推进循环，那它通常只是 LLM application，不是典型 agent。

当然，如果这个 chatbot 内部会自动检索、调用工具、判断是否继续执行、持续处理上下文，那它就开始具有 agent 特征了。也就是说，界面像聊天不决定是不是 agent，关键看内部运行机制。

2. 固定流程的 workflow 不一定是 agent

假设有一个程序：先做意图分类，再检索文档，再让模型总结，再输出答案。每一步都是预定义的，没有动态决定下一步的能力，那么它更像 workflow。

很多面试官喜欢问：Workflow 算不算 agent？比较稳妥的答法是：

- 广义上，有些人会把带一点动态性的工作流也叫 agentic workflow；
 - 但严格区分时，workflow 侧重预先定义路径，agent 侧重运行时自主决策。
- 这种回答既承认现实里术语会混用，又保留了工程上的清晰边界。

3. 只有 function calling 不足以自动成为 agent

如果系统只是让模型根据 schema 选择一个函数，然后程序执行后直接结束，这更像 tool-augmented LLM。

只有当它具备多步循环、根据工具结果持续决策的能力，才更接近典型 agent。

换句话说：**工具是 agent 的常见组成，不是充要条件。**

4. 有记忆不等于 agent

有些应用会保存用户画像、会话历史、偏好设置，但如果没有自主行动和多步执行，仍然不一定是 agent。

Memory 解决的是状态延续问题，agent 解决的是目标推进问题。两者经常同时出现，但不是一回事。

四、为什么 Agent 会火：它到底解决了什么问题

理解 agent 的价值，不能只说更智能。更准确的说法是：它解决的是**固定流程难以覆盖的开放式任务执行问题**。

场景一：任务路径事先不确定

例如帮我调研最近一周行业新闻并写成分析，到底先查哪几个网站、要不要改查询词、哪些信息要交叉验证，这些步骤很难完全提前写死。

Agent 的价值在于让模型在运行过程中适配具体情况。

场景二：任务需要多轮环境交互

比如电脑操作、浏览网页、调用多个系统 API、管理文件、运行代码、读取执行结果、基于结果继续操作。

这种场景不是靠一次文本生成能完成的，而是需要动作—观察—再动作的闭环。

场景三：任务规模超过单轮上下文

例如复杂研究、长文档生成、跨多个数据源汇总、长期追踪式任务。
此时需要规划、拆解、记忆、阶段性产物存储，agent 由此变得更合适。

场景四：任务要求一定程度的自动化闭环

当系统需要从给建议升级为帮你完成，agent 的意义就上来了。
比如客服知识问答可能只需要 RAG；但如果要识别退款诉求—调订单系统—核验规则—发起审批—通知用户，那就明显更接近 agent。

五、Agent 的核心构成：面试一定要会讲的最小框架

真正回答什么是 agent，最稳妥的方法不是背论文，而是给出一个最小系统框架。一个生产可理解的 agent，通常至少包括下面这些部分：

1. Goal / Task

系统到底要完成什么目标。
目标可以来自用户，也可以来自上层程序分配。目标需要尽可能明确边界，例如成功标准、限制条件、预算、时限、权限范围。

2. Policy / Planner

由模型或规则决定下一步做什么。
有时是即时反应式决策（如 ReAct），有时是先生成计划再执行（如 Plan-and-Execute），有时还会混入程序化策略。

3. Tools / Environment

外部能力与外部世界接口。
包括搜索、数据库、代码执行、浏览器、办公系统、业务系统 API、设备控制等。没有外部环境交互，很多 agent 价值发挥不出来。

4. State / Memory

运行时状态、历史观测、阶段结果、用户偏好、任务上下文。

这部分决定 agent 是否能在多步执行中保持一致性，也决定是否支持长任务恢复。

5. Guardrails / Governance

权限、审批、参数校验、风险控制、日志审计、终止条件。

这是校园面试里最容易漏掉，但工程上最关键的部分。真正上线的 agent，不可能只靠模型自己判断。

6. Evaluator / Verifier

结果校验器或中间步骤检查机制。

例如检查是否回答完整、是否满足格式、是否存在自相矛盾、是否命中了业务规则。

很多 agent 失败，不是因为不会生成，而是因为没人验证。

把这六个部件讲清楚，面试官基本就能判断你是不是理解了 agent 的系统本质。

六、Agent 的自治程度：不要把全自动当成唯一答案

另一个高频追问是：Agent 一定要完全自治吗？

标准回答是：不一定。自治是连续谱，不是二元开关。

可以按自治程度粗略分成四类：

1. Assisted Agent

以人类为主，模型提供建议，工具执行仍然依赖人触发。

比如 IDE 里的代码建议、审批流中的推荐决策。

2. Semi-Autonomous Agent

大部分步骤可自动执行，但关键节点需要人工确认。

比如可自动检索和草拟邮件，但发送前需要审批；可自动起草工单处理方案，但涉及退款或删库操作必须人工确认。

3. Autonomous Agent

在给定权限边界内可独立完成较长链路任务。

例如定期市场情报收集、自动化测试执行、基础级别的报表整理与推送。

4. Open-Ended Agent

目标开放、环境复杂、时间跨度长，可能长期运行。

这是研究和前沿系统更关注的方向，但在企业场景里通常最难治理。

面试里很加分的一句话是：自治程度越高，对权限、状态、审批、终止条件和评测体系的要求越高。

这样回答能体现你不是在崇拜自动化，而是在做系统权衡。

七、Agent 适合什么场景，不适合什么场景

适合的场景

1. 路径不固定、需要探索

例如复杂研究、网页操作、多源信息搜集、跨系统流程执行。

这类任务很难穷举所有路径，用 agent 能减少硬编码。

2. 明显依赖中间观察结果

工具返回什么，直接影响下一步怎么做。

比如网页某个按钮不存在、API 某个字段缺失、数据库返回空结果，都需要动态调整。

3. 任务可以被拆成阶段，但阶段之间仍需动态协调

比如先搜集信息，再筛选，再归纳，再输出报告，每个阶段都可能因中间结果而调整。

4. 任务价值高于控制成本

如果为了自动化一个小任务，引入 agent 会带来大量复杂性，那不值得。

Agent 更适合那些单次成功收益较高、人工成本较高、流程变体很多的任务。

不适合的场景

1. 路径清晰且稳定

例如固定报表生成、标准审批流、简单表单填写。

这些任务用 workflow 更简单、更稳、更便宜。

2. 错误代价极高但又难以完全约束

例如高风险金融交易、关键生产控制、无保护措施破坏性操作。
不是说绝对不能做，而是要极其谨慎，通常必须加强规则、审批与沙箱。

3. 数据与接口不稳定、但系统又不能容错

Agent 高度依赖工具和环境反馈，如果底层系统本身非常不稳定，agent 只会把问题放大。

4. 只是为了看起来高级

这是面试里经常会遇到的陷阱。如果一个需求明明两段规则 + 一次检索就能解决，却硬做成多 agent，那会显得你缺乏成本意识。

九、常见误区与纠偏

误区一：用了工具就是 agent

纠偏：工具增强型应用和 agent 的区别在于是否存在目标推进型运行循环，以及是否根据观察持续决策。

误区二：agent 一定比 workflow 高级

纠偏：工程上没有高级一说，只有是否合适。很多场景 workflow 更优。

误区三：agent 就是把 prompt 写复杂一点

纠偏：真正的 agent 设计重点不只在 prompt，而在状态、工具、约束、监控与恢复机制。

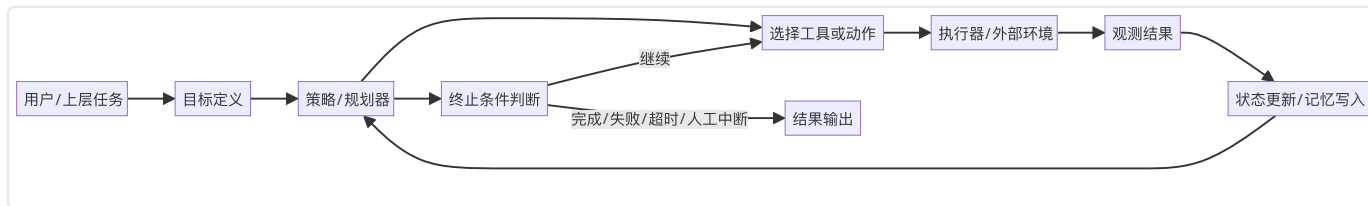
误区四：只要模型够强，就不需要系统设计

纠偏：模型再强，也无法替代权限管理、幂等控制、审批机制、日志审计和终止条件。

误区五：agent 的核心是思维链

纠偏：推理过程只是其中一部分。很多真实系统的关键瓶颈反而在工具可靠性、上下文管理和任务边界定义。

十、Agent 的最小运行闭环



十一、高频面试题与参考回答

题 1：什么是 Agent？和普通 LLM 应用有什么不同？

参考回答：

Agent 是围绕目标进行多步决策和动作执行的系统，不是单次文本生成。普通 LLM 应用更像输入问题，输出答案；Agent 更像给定目标，自动选择步骤、调用工具、根据观察继续执行，直到终止。

题 2：带 function calling 的聊天机器人算不算 agent？

参考回答：

不一定。如果只是一次性选择一个工具然后结束，更像 tool-augmented chatbot；如果系统会根据工具结果持续决策、多步推进任务，那就更接近 agent。

题 3：Workflow 和 Agent 的区别是什么？

参考回答：

Workflow 的路径更多是预定义的，强调稳定、可控、可回放；Agent 的路径在运行时动态决定，适合任务不确定性更高的场景。

题 4：Agent 一定需要工具吗？

参考回答：

从广义上说，不一定；但从高价值的工程应用看，几乎都需要外部能力。没有工具，agent 很难突破模型参数内知识的限制，也很难真正完成任务。

题 5：什么时候你不会建议上 Agent？

参考回答：

任务路径清晰、规则强、结果必须稳定可控、错误代价高且可通过规则系统解决时，我更倾向于 workflow 或纯程序逻辑。

题 6：为什么大家总说 agent 难做？

参考回答：

难点不在于让模型想一步，而在于如何在复杂环境里持续正确做事。包括工具可靠性、上下文污染、权限控制、长任务恢复、终止条件、评测与回放等。

题 7：Agent 的核心组件有哪些？

参考回答：

目标、策略/规划、工具、状态/记忆、控制约束、结果校验。这六块缺一不可，尤其在生产环境里。

题 8：Agent 和 RAG 的关系是什么？

参考回答：

RAG 是一种知识获取机制，解决从外部知识源取信息；Agent 是任务执行范式，解决如何基于目标多步推进。RAG 可以是 agent 的一个工具或子模块。

题 9：Agent 和多 agent 是一回事吗？

参考回答：

不是。多 agent 只是 agent 系统的一种组织方式。很多场景单 agent 足够，而且更稳。多 agent 只有在上下文隔离、角色专门化、并行化或团队分工价值明显时才值得上。

题 10：怎样判断一个需求是否应该 agent 化？

参考回答：

看三点：任务路径是否难以预先编码；是否需要多轮环境交互；自动化收益是否足以覆盖引入自治后的复杂度和风险。

十二、面试速记版

- Agent 的关键不是能聊天，而是能围绕目标持续行动。
- Agent 的关键特征：目标驱动、多步决策、环境交互、状态持续。
- 工具不是充分条件；循环式决策才是核心。
- Workflow 强在稳定和可控，Agent 强在处理不确定路径。
- 真正的 Agent 设计必须补齐权限、审批、日志、恢复、终止条件。

- 不是所有任务都值得 agent 化，过度 agent 化是常见反模式。

参考资料与延伸阅读

1. Anthropic, 《Building effective agents》, 2024。适合用来界定 agent 与 workflow 的边界，强调能用简单工作流解决时不要过度 agent 化。
2. Anthropic, 《Effective context engineering for AI agents》, 2025。非常适合补充 agent 不只是 prompt engineering, 而是 context engineering 的近年观点。
3. Anthropic, 《How we built our multi-agent research system》, 2025。适合放在多 agent、任务拆解、subagent 协作章节中。
4. OpenAI, 《A practical guide to building agents》, 2025。适合用作工具类型、agent 运行循环、单 agent 优先、多 agent 何时有意义的工程基线。
5. LangChain / LangGraph 官方文档, 《Workflows and agents》《Multi-agent》《Human-in-the-loop》《Interrupts》《Durable execution》。
6. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2023。
7. Wang et al., *Executable Code Actions Elicit Better LLM Agents (CodeAct)*, 2024。
8. Yao et al., *Tree of Thoughts*, 2023。
9. Kim et al., *An LLM Compiler for Parallel Function Calling*, 2024。
10. Erdogan et al., *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks*, 2025。
11. Schick et al., *Toolformer*, 2023。
12. Packer et al., *MemGPT*, 2024。
13. Park et al., *Generative Agents*, 2023。
14. Shinn et al., *Reflexion*, 2023。
15. Wu et al., *AutoGen*, 2023。
16. Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, 2025。适合补充多 agent 失败模式。
17. Mialon et al., *GAIA: A Benchmark for General AI Assistants*, 2023。适合在为什么 agent 需要工具、规划、执行时做 benchmark 背景延伸。